

Software Defined Wireless Network Testbed using Raspberry Pi OF Switches with Routing Add-on

Mehmet Ariman, Gökhan Seçinti, Müge Erel, and Berk Canberk

Department of Computer Engineering,

Istanbul Technical University,

Ayazaga, 34469, Istanbul - TURKEY

E-mail: {ariman, secinti, erelmu, canberk}@itu.edu.tr

Abstract—In this demo, we implement a realtime testbed for Software Defined Wireless Networks (SDWN) by using Raspberry Pi as OpenFlow (OF) Switches. The implemented testbed provides practical development and testing environment for SDWNs. Moreover, we use OpenDayLight to observe the flows and events in the network. With OpenDayLight integration, we easily provide detailed analysis results for any testing process. Additionally, we develop a traffic aware routing algorithm add-on which manages the flows w.r.t. their QoS requirements. Finally, we physically implement the proposed routing algorithm to our testbed and validate the improvements of traffic aware routing such as controller response time and network utilization.

Keywords—SDWN, Raspberry Pi, OpenFlow Switches, Routing

I. INTRODUCTION

The introduction of the SDWN has dramatically changed the way of manageability of current network resources and provided new perspectives for researchers to innovate current network protocols. To this end, it is critical to provide a resourceful testbed which is also inexpensive and easy-to-implement in order to supply the academia with the necessary development and testing environment. In this demo, we provide a practical testbed for SDWN where OF switches are realized by using Raspberry Pi with WiFi dongles. We also implement a traffic aware routing algorithm add-on in order to test and configure the testbed environment. Before giving the implementation details which are covered in Section II, we give a literature survey about current SDN implementations by emphasizing our demo contributions, in the following subsection.

A. Related Work

In the study [1], the authors have provided an extension to OpenFlow which focuses on race conditions. They have used a model checking verifier in order to validate the performance of their extension. The mobile-oriented service slicing method has been proposed to optimize the usage of limited resources for Mobile Networks. A network-fabric based model has been proposed in [2]. The proposed model replaced the flow loop-up mechanism in OF switches to improve the response times. However, the performance of these two studies have been validated by using Mininet Emulation rather than by using real hardware implementations. In the study [3], the authors have also implemented OF switches by using

Raspberry Pi hardware. Additionally, they have compared the network throughput performance of the implemented switch with the NetFPGA-1G switch. However, the implemented hardware only tackles wired connection as the access network and considers wireless connections as a future work. More importantly this implementation uses OpenFlow v1.0 which does not support QoS aware flow management.

To the best of our knowledge, there are very limited studies which implements OF switches using existing hardware in the literature. To this end, we focus on implementing OF switch with the integration of OpenDaylight. We use OpenFlow v1.3.0 which has the additional capabilities such as Layer 3 and Layer 4 matching, flexible filtering. Moreover, we have proposed additional routing algorithm to validate the implemented infrastructure. As main contribution, we provide easy-to-implement testbed for SDN and also to shown validation of the proposed routing algorithm by using the testbed provided.

II. IMPLEMENTATION

A. Open-Flow Switch

Open-Flow Switches are defined as the software implementation of specialized network hardware features on inexpensive general purpose hardware to be used in different layers of the deployed network topology. Furthermore, Raspberry Pi hardware platform described in [4] is picked to replace the legacy network hardware. In addition, the number of ports and physical mediums on the device is designed to be extended by USB ports placed on the platform using external USB devices such as USB to Ethernet converter and Wi-Fi dongle.

The software stack of the implemented switch is based on widely used Open Source Softwares to reduce implementation time and effort. OpenWRT [5] lies in the heart of the stack and the latest release of the software is used as the base embedded Linux distribution. The existing support for Raspberry Pi in distribution also reduced the implementation cost. The distribution is modified to enhance USB to Ethernet converters and Realtek 5370 Wi-Fi dongles by adding support of related device drivers. The device is designed to act as Soft Access Point to allow hosts to communicate through wireless medium. Furthermore, hostapd open source software is ported and installed to the switch implementation. Finally, the device is converted into an OpenFlow Switch by porting 2.3.90 version of OpenVSwitch (OVS) REF-OpenVSwitch-Website which support OpenFlow 1.3 specs to our platform to collaborate with

the operating system running on the device which is Linux Kernel 3.18.16. Linux Kernel together with OVS provides the service chaining infrastructure required by SDN Controller to implement proposed solution.

B. Controller

The proposed solution requires centralized SDN controller placed in the cloud which communicates with the network devices explained in II-A via OpenFlow and implements virtual network functions (VNFs). Furthermore, an Open Source SDN Controller is installed on a server computer. The Helium Release of OpenDaylight Controller [6] is used as SDN Controller which already includes the required southbound implementation to communicate with the OpenFlow Switch implemented in II-A. The OpenDaylight also provides a nice network topology viewer which also visualizes the flow tables and the flow statistics of the OpenFlow Switches orchestrated by the controller. In addition, OSGI based northbound implementations included in OpenDaylight provides the VNFs required to orchestrate the network. We have implemented and integrated the proposed routing algorithm as northbound service application to the OpenDaylight.

III. PLANNED DEMO

The system architecture depicted in the Figure 1 is used for demonstration. OpenFlow (OF) switches used to build network infrastructure will communicate with the controller via OpenFlow 1.3 [7]. In addition, the traffic between the switches and the server will be monitored using Wireshark on controller. Moreover, while hosts will connect to the access devices, OF Switches, using wireless medium, OF Switches and the controller will be connected among themselves through the wired medium.

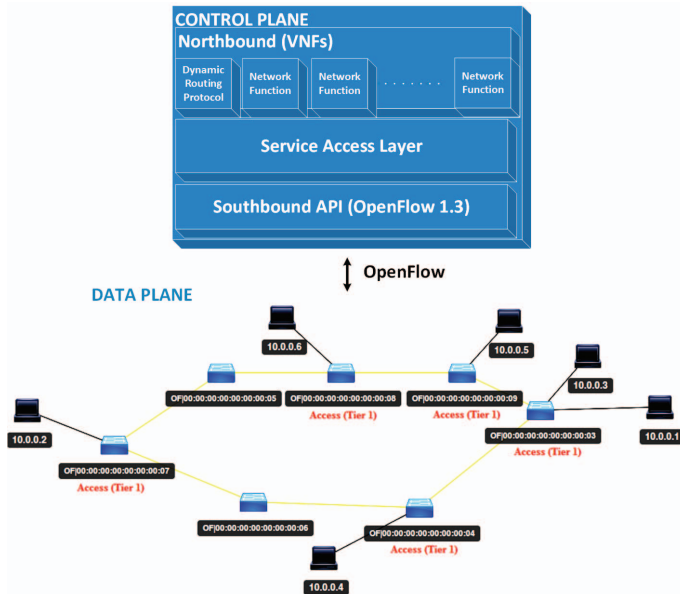


Fig. 1. Network Architecture

The controller software will depict the overall view of the network topology including connected hosts. Furthermore,

Node	In Port	DL Src	DL Dst	DL Type	DL Vlan	NW Src	NW Dst	NW Proto	TP Src	TP Dst	Actions	Byte Count	Packet Count	Duration Seconds	Idle Timeout	Priority
OF:00:00:00:00:00:00:04	*	*	*	IPv4	*	10.0.0.2	*	*	*	*	OUTPUT = OF:3	0	0	57	0	1
OF:00:00:00:00:00:00:04	*	*	*	IPv4	*	10.0.0.6	*	*	*	*	OUTPUT = OF:2	0	0	57	0	1
OF:00:00:00:00:00:00:04	*	*	*	IPv4	*	10.0.0.4	*	*	*	*	SET_DL_DST = 00:00:00:00:00:04 OUTPUT = OF:1	1078	11	57	0	1
OF:00:00:00:00:00:00:04	*	*	*	IPv4	*	10.0.0.3	*	*	*	*	OUTPUT = OF:2	0	0	57	0	1

Fig. 2. Flow Table of the OF Switch 00:00:00:00:00:00:04

this topology will be accessible through web browser on the controller. Traffic conditions aforementioned below will be created and the flow of the traffic will also be tracked by viewing the flow tables of the OF Switches using the web browser. OpenDaylight provides an interface in order to observe the detail of the network infrastructure and behaviour. The OF Switch hardware forwards the packets received from the data plane interfaces using the flow table entries that is generated and placed by the controller. Flow table entries match the packets using the Data Link Layer information, Source MAC address and Destination MAC address, Network Layer information, source IP Address, destination IP Address and the transport layer protocol, Transport Layer information, source port and destination port, and the input interface. The sample snapshot of a flow table of an OF Switch is given in the Figure 2. The contents of the table are acquired by using OpenDaylight.

Furthermore, the characteristics of the proposed routing algorithm will be demonstrated for the following scenarios:

- Routing of the different type of traffic, such as RTP and FTP, between same hosts.
- Routing under different traffic loads.

The proposed routing add-on aims to improve the network utilization by maintaining heterogeneous traffic types for the underlying users. Consequently, we implement the routing add-on to the developed testbed and validate the improvement in network channel usage.

REFERENCES

- [1] X. Sun, A. Agarwal, and T. Ng, "Controlling race conditions in openflow to accelerate application verification and packet forwarding," *IEEE Transactions on Network and Service Management*, vol. 12, no. 2, pp. 263–277, June 2015.
- [2] M. Martinello, M. Ribeiro, R. de Oliveira, and R. de Angelis Vitoi, "Keyflow: a prototype for evolving sdn toward core network fabrics," *IEEE Network*, vol. 28, no. 2, pp. 12–19, March 2014.
- [3] H. Kim, J. Kim, and Y.-B. Ko, "Developing a cost-effective openflow testbed for small-scale software defined networking," in *2014 16th International Conference on Advanced Communication Technology (ICACT)*, Feb 2014, pp. 758–761.
- [4] "Raspberry pi hardware specifications," Tech. Rep., 2011. [Online]. Available: <https://www.raspberrypi.org/documentation/hardware/>
- [5] "Developer's guide - basic approach to openwrt," OpenWrt - Wireless Freedom, Tech. Rep., 2012. [Online]. Available: <http://wiki.openwrt.org/doc/techref/externaldocumentation>
- [6] "OpenDaylight - an open source community and meritocracy for software defined networking," Linux Foundation, Collaborative Projects, Tech. Rep., 2013.
- [7] "Openflow switch specification version 1.3.0," ONF, Open Networking Foundation, Tech. Rep., 2012.